

Week 7: Simultaneous Equation Models

Samuel Rowe adapted from Wooldridge (2011) and Mitchell (2020)

2025-09-11

Contents

Overview	5
1 Simultaneous Equation Models and Mitchell Chapter 9	7
1.1 Labor Supply of Married, Working Women	7
1.2 Inflation and Openness	10
1.3 Exercises	11
2 Appending and Merging	13
3 Appending	15
3.1 Appending Datasets	15
3.2 Appending Problems	19
3.3 Merging: One-to-one	31
3.4 Merging One-to-many	37
3.5 Merging multiple datasets	43
3.6 Merging Problems	50
3.7 Joining datasets	55
3.8 Crossing datasets	58

bookdown::render_book()

Overview

This week we will cover the implementation of simultaneous equation models for econometric analysis. We will also cover appending and merging for data management.

1. Simultaneous Equation Models
2. Appending
3. Merging

Chapter 1

Simultaneous Equation Models and Mitchell Chapter 9

1.1 Labor Supply of Married, Working Women

Let's look at the data for married working women

```
cd "/Users/Sam/Desktop/Econ 645/Data/Wooldridge"  
use mroz.dta, clear
```

Our labor supply equation

$$hours_i = \beta_{10} + \alpha_1 \ln(wage_i) + \beta_{11} educ_i + \beta_{12} age_i + \beta_{13} kidslt6_i + \beta_{14} nwifeinc + u_1$$

Our labor demand equation in term of wages as a function of productivity

$$\ln(wage_i) = \beta_{20} + \alpha_2 hours_i + \beta_{21} * educ_i + \beta_{22} exper_i + \beta_{23} exper_i^2 + u_2$$

We will use the *ivregress 2sls* command to estimate our labor supply we use educ, age, kidslt6, nwifeinc, exper, and exper-squared.

```
ivregress 2sls hours (lwage=exper expersq) educ age kidslt6 nwifeinc, robust
```

Instrumental variables (2SLS) regression	Number of obs	=	428
	Wald chi2(5)	=	12.60

```

Prob > chi2      =      0.0274
R-squared        =      .
Root MSE        =     1344.7

```

			Robust				
hours	Coef.	Std. Err.	z	P> z	[95% Conf. Interval]		
lwage	1639.556	593.3108	2.76	0.006	476.6879	2802.423	
educ	-183.7513	67.78742	-2.71	0.007	-316.6122	-50.89039	
age	-7.806092	10.48746	-0.74	0.457	-28.36114	12.74896	
kidslt6	-198.1543	208.4247	-0.95	0.342	-606.6592	210.3506	
nwifeinc	-10.16959	5.287486	-1.92	0.054	-20.53287	.1936911	
_cons	2225.662	603.0964	3.69	0.000	1043.615	3407.709	

Instrumented: lwage

Instruments: educ age kidslt6 nwifeinc exper expersq

Test exper and expersq as instruments

```

reg lwage exper expersq educ age kidslt6 nwifeinc, robust
test exper expersq

```

```

Linear regression      Number of obs      =      428
                      F(6, 421)          =      14.76
                      Prob > F            =      0.0000
                      R-squared           =      0.1633
                      Root MSE          =      .66622

```

			Robust				
lwage	Coef.	Std. Err.	t	P> t	[95% Conf. Interval]		
exper	.0418643	.0151135	2.77	0.006	.0121569	.0715718	
expersq	-.0007625	.0004065	-1.88	0.061	-.0015614	.0000365	
educ	.1011113	.0141358	7.15	0.000	.0733257	.128897	
age	-.0025561	.0059149	-0.43	0.666	-.0141826	.0090704	
kidslt6	-.0532185	.1047899	-0.51	0.612	-.2591951	.1527581	
nwifeinc	.00556	.0027435	2.03	0.043	.0001673	.0109527	
_cons	-.4471607	.2889008	-1.55	0.122	-1.015028	.1207069	

- (1) exper = 0
(2) expersq = 0

$$\begin{array}{rcl} F(2, 421) & = & 6.17 \\ \text{Prob} > F & = & 0.0023 \end{array}$$

Note: Our instrument is rather weak. What is a possible problem with this first-stage?

Interpretation: Our results show that the labor supply curve slopes upward. The estimated coefficient is

$$\Delta hours = 1640/100 * (\% \Delta wages)$$

Our results can be interpreted through linear-log elasticity.

$$100 * (\Delta hours / hours) \approx (1,640 / hours) (\% \Delta wages)$$

Or

$$\% \Delta hours \approx (1,640 / hours) (\% \Delta wages)$$

This implies that the labor supply elasticity with respect to wages is

$$\eta = 1,640 / hours$$

The elasticity is not constant since it is a linear-log model instead of log-log model. Using average hours worked of 1,303 Our estimated elasticity is

$$(1,640 / hours) (\% \Delta wages) = 1,640 / 1,303 = 1.26$$

`display 1640/1303`

1.2586339

Our estimated elasticity around mean hours is 1.26%, which means a 1% increase in wages around mean hours increases hours by 1.26%. This mean wage elasticity of supply is elastic (>1) around mean hours.

With a linear-log model, the estimated elasticity is not constant elasticity.

When hours are higher, full time 40 hours every week, a 1% increase in wages increase the supply of hours around 0.79%

`display 1640/(40*52)`

.78846154

Our wage elasticity of supply is inelastic since (<1)

At lower hours, our estimated wage elasticity of supply is more elastic. When hours are equal to 800, then our wage elasticity of supply is almost 2

```
display 1640/800
```

2.05

A 1% increase in wages increases hours by 2.05%

1.2 Inflation and Openness

Romer (1993) proposes that more open countries should have lower inflation rates. Romer (1993) tries to explain inflation rates in terms of a country's average share of imports in gross domestic (or national) product since 1973. Average share of imports in GDP is his measure of openness.

```
cd "/Users/Sam/Desktop/Econ 645/Data/Wooldridge"
use openness.dta, clear
```

$$\begin{aligned} inf_{i,t} &= \beta_{10} + \alpha_1 open_{i,t} + \beta_{11} \ln(pcinc_{i,t}) + u_1 \\ open_{i,t} &= \beta_{20} + \alpha_2 inf_{i,t} + \beta_{21} \ln(pcinc)_{i,t} + \beta_{22} \ln(land)_{i,t} + u_2 \end{aligned}$$

Where open is the average share of imports in terms of GDP, pcinc is the 1980 per capita income, and land is land area of a country in square miles

We can look at the reduced form equation to see the instruments impact on openness.

```
reg open lland lpcinc, robust
```

Linear regression	Number of obs	=	114
	F(2, 111)	=	22.22
	Prob > F	=	0.0000
	R-squared	=	0.4487
	Root MSE	=	17.796

		Robust				
open	Coef.	Std. Err.	t	P> t	[95% Conf. Interval]	
-----+-----						

```

      lland | -7.567103   1.141798   -6.63   0.000   -9.829652   -5.304554
    lpcinc |  .5464812   1.436115    0.38   0.704   -2.299276    3.392238
      _cons |  117.0845   18.24808    6.42   0.000    80.92473   153.2443
-----

```

Let's test the instrument and use an F-test.

```
test lland
```

```

( 1)  lland = 0

      F( 1, 111) =   43.92
      Prob > F =   0.0000

```

We want to see if a country's openness impacts a country's inflation rate using natural log of square miles as an instrument.

```
ivregress 2sls inf (open=lland) lpcinc, robust
```

```

Instrumental variables (2SLS) regression      Number of obs   =       114
                                              Wald chi2(2)    =        5.19
                                              Prob > chi2     =       0.0745
                                              R-squared      =       0.0309
                                              Root MSE      =       23.52

```

```

-----+-----
            |               Robust
      inf |      Coef.   Std. Err.      z    P>|z|     [95% Conf. Interval]
-----+-----
      open |  -.3374871   .1504296    -2.24   0.025   - .6323238   -.0426504
    lpcinc |   .3758247   1.360282     0.28   0.782   -2.29028    3.041929
      _cons |  26.89934   10.77526     2.50   0.013    5.780212   48.01846
-----+-----

```

```

Instrumented:  open
Instruments:   lpcinc lland

```

Interpretation: For every percentage point increase in average share of imports of GDP, inflation decreases by 0.337 percentage points

1.3 Exercises

On ELMS, pull CPS data and recreate MROZ.dta with CPS data, expect use all individuals not just working women

```
use smalljul25pub.dta, clear
```

Estimate linear-log and test elasticity at different hours. Estimate constant elasticity with a log-log model.

Chapter 2

Appending and Merging

Appending and merging datasets are a crucial part of learning Stata or any other statistical package. Many times we are merging datasets by State FIPS, year, Zip Codes, Unit ID, County FIPS, etc. If we wanted to analyze local-level unemployment data to our analysis of county-level crime, then we will likely be using data from BLS and data from FBI or other local admin data.

Note: Working with multiple datasets: I do not have frames in Stata 14, but using frames is encouraged. Using the temp files workaround is a bit cumbersome (but it works). Using Stata frames will help work with those datasets simultaneously to get them ready to merge.

Chapter 3

Appending

The *append* command appends two or more datasets into one data set. We are appending rows with the same columns. Appending is more straightforward than merging, but still an important tool for examining multiple datasets.

3.1 Appending Datasets

Let's say we have two datasets with the same variables. We can append these files together using the *append* command.

```
cd "/Users/Sam/Desktop/Econ 645/Data/Mitchell"
use "moms.dta", clear
list
use "dads.dta", clear
list
```

/Users/Sam/Desktop/Econ 645/Data/Mitchell

	famid	age	race	hs
1.	3	24	2	1
2.	2	28	1	1
3.	4	21	1	0
4.	1	33	2	1

	famid	age	race	hs
--	-------	-----	------	----

	famid	age	race	hs
1.	1	21	1	0
2.	4	25	2	1
3.	2	25	1	1
4.	3	31	2	1

We can append the mom.dta file with *append* using filename.

```
append using "moms.dta"
list
```

	famid	age	race	hs
1.	1	21	1	0
2.	4	25	2	1
3.	2	25	1	1
4.	3	31	2	1
5.	3	24	2	1
6.	2	28	1	1
7.	4	21	1	0
8.	1	33	2	1

Or,

```
clear
append using "moms.dta" "dads.dta"
list
```

	famid	age	race	hs
1.	3	24	2	1
2.	2	28	1	1
3.	4	21	1	0
4.	1	33	2	1
5.	1	21	1	0
6.	4	25	2	1
7.	2	25	1	1
8.	3	31	2	1

Note: What is a clear problem here? After the append, how do we identify which data are for dads and for moms?

There are two ways. 1. We can generate a variable in both files and code it. 2. We can use the *generate* option in the append.

```
clear
append using "moms.dta" "dads.dta", generate(datascreen)
list, sepby(datascreen)
```

	datasc~n	famid	age	race	hs
1.	1	3	24	2	1
2.	1	2	28	1	1
3.	1	4	21	1	0
4.	1	1	33	2	1
5.	2	1	21	1	0
6.	2	4	25	2	1
7.	2	2	25	1	1
8.	2	3	31	2	1

Since moms.dta is first, the new variable datascreen will set moms.dta datascreen variable = 1, and since dads.dta is second, the datascreen variable = 2.

You could just generate a variable called parent in both files and set moms equal to 1 and dads equal to 0. But, the *generate* option is nice and concise.

It's a good idea to label the data

```
label define datascreenl 1 "From moms.dta" 2 "From dads.dta"
label values datascreen datascreenl
list, sepby(datascreen)
```

	datascreen	famid	age	race	hs
1.	From moms.dta	3	24	2	1
2.	From moms.dta	2	28	1	1
3.	From moms.dta	4	21	1	0
4.	From moms.dta	1	33	2	1
5.	From dads.dta	1	21	1	0
6.	From dads.dta	4	25	2	1
7.	From dads.dta	2	25	1	1

```

8. | From dads.dta      3    31      2    1 |
+-----+

```

If we use the *generate* option in *append* with one file open, the values of the generated variable are different.

```

clear
use "moms.dta"
append using "dads.dta", generate(datascreen)
list, sepby(datascreen)
label define datascreen1 0 "From moms.dta" 1 "From dads.dta"
label values datascreen datascreen1
list, sepby(datascreen)

```

```

      | famid   age   race   hs   datascreen |
+-----+
1. |      3    24      2    1           0 |
2. |      2    28      1    1           0 |
3. |      4    21      1    0           0 |
4. |      1    33      2    1           0 |
+-----+
5. |      1    21      1    0           1 |
6. |      4    25      2    1           1 |
7. |      2    25      1    1           1 |
8. |      3    31      2    1           1 |
+-----+

```

```

+-----+
      | famid   age   race   hs   datascreen |
+-----+
1. |      3    24      2    1   From moms.dta |
2. |      2    28      1    1   From moms.dta |
3. |      4    21      1    0   From moms.dta |
4. |      1    33      2    1   From moms.dta |
+-----+
5. |      1    21      1    0   From dads.dta |
6. |      4    25      2    1   From dads.dta |
7. |      2    25      1    1   From dads.dta |
8. |      3    31      2    1   From dads.dta |
+-----+

```

We can append multiple datafiles together (as long as they have the same variables).

```

dir br*.dta
use "br_clarence.dta", clear
list

clear
append using "br_clarence.dta" "br_isaac" "br_sally", generate(rev)
label define revl 1 "clarence" 2 "isaac" 3 "sally"
label values rev revl
list, sepby(rev)

```

```

-rw-r--r-- 1 Sam staff 2604 Aug 15 2023 br_clarence.dta
-rw-r--r-- 1 Sam staff 2553 Aug 15 2023 br_isaac.dta
-rw-r--r-- 1 Sam staff 2583 Aug 15 2023 br_sally.dta

```

```

+-----+
| booknum                                book  rating |
+-----+
1. |      1          A Fistful of Significance          5 |
2. |      2  For Whom the Null Hypothesis is Rejected    10 |
3. |      3  Journey to the Center of the Normal Curve    6 |
+-----+

```

```

+-----+
|      rev  booknum                                book  rating |
+-----+
1. | clarence      1          A Fistful of Significance          5 |
2. | clarence      2  For Whom the Null Hypothesis is Rejected    10 |
3. | clarence      3  Journey to the Center of the Normal Curve    6 |
+-----+
4. | isaac         1          The Dreaded Type I Error            6 |
5. | isaac         2          How to Find Power                    9 |
6. | isaac         3          The Outliers                         8 |
+-----+
7. | sally         1          Random Effects for Fun and Profit    6 |
8. | sally         2          A Tale of t-tests                    9 |
9. | sally         3  Days of Correlation and Regression            8 |
+-----+

```

3.2 Appending Problems

We can check the two datasets for potential problems with the *precombine* command. You will need to install this community-user command. Using *precombine* with the *describe* option, we can check to see if the components of the

datasets are similar to prevent problem: 1. Storage type 2. Format 3. Variable labels 4. Values labels 5. Type of variable - string or numeric 6. Position of the variable.

```
search precombine
clear
precombine "moms.dta" "dads.dta", describe(type format varlab vallab ndta) uniquevars
```

request ignored because of batch mode

Reports relevant to the combining of the following datasets:

```
[vars: 4    obs: 4]      moms.dta
[vars: 4    obs: 4]      dads.dta
```

Variables that appear in multiple datasets:

variable	dataset	type	format	varlab	vallabname	ndta
age	dads.dta	float	%5.0g	Age		2
age	moms.dta	float	%5.0g	Age		2
famid	dads.dta	float	%5.0g	Family ID		2
famid	moms.dta	float	%5.0g	Family ID		2
hs	dads.dta	float	%7.0g	HS Graduate?		2
hs	moms.dta	float	%7.0g	HS Graduate?		2
race	dads.dta	float	%5.0g	Race		2
race	moms.dta	float	%5.0g	Ethnicity		2

There are no variables that appear in only one dataset,
i.e. every variable appears in multiple datasets.

3.2.1 Different variable names

If the same variable intent has two different variable names between the data sets, then they will become two different columns when you only want one column of data.

For example, when we append moms1 and dads1 which have different variable names for the different variables, the variables do not append correctly.

```
use "moms1.dta", clear
append using "dads1.dta", generate(datascreen)
list
```

	famid	mage	mrace	mhs	datasc~n	dage	drace	dhs
1.	1	33	2	1	0	.	.	.
2.	2	28	1	1	0	.	.	.
3.	3	24	2	1	0	.	.	.
4.	4	21	1	0	0	.	.	.
5.	1	.	.	.	1	21	1	0
6.	2	.	.	.	1	25	1	1
7.	3	.	.	.	1	31	2	1
8.	4	.	.	.	1	25	2	1

Solution

It's an easy fix. Just rename the variables to a common variable name and append.

```
use "moms1.dta", clear
rename (mage mrace mhs) (age race hs)
save "moms1temp.dta", replace

use "dads1.dta", clear
rename (dage drace dhs) (age race hs)
save "dads1temp.dta", replace

append using "moms1temp.dta" "dads1temp.dta", generate(datascreen)
list
```

file moms1temp.dta saved

file dads1temp.dta saved

	famid	age	race	hs	datasc~n
1.	1	21	1	0	0
2.	2	25	1	1	0
3.	3	31	2	1	0

4.		4	25	2	1	0	
5.		1	33	2	1	1	

6.		2	28	1	1	1	
7.		3	24	2	1	1	
8.		4	21	1	0	1	
9.		1	21	1	0	2	
10.		2	25	1	1	2	

11.		3	31	2	1	2	
12.		4	25	2	1	2	
+-----+							

3.2.2 Conflicting variable labels

If we have conflicting variable labels, then the variable labels of the primary dataset will overwrite the appended dataset. The solution is to use a neutral variable label. For example, instead of Mom's HS and Dad's HS, just have "high school" as the variable label in both datasets

Solution: For appending, use neutral variable label names.

```
use "momslab.dta", clear
append using "dadslab.dta", generate(datascreen)
describe

use "momslab.dta", clear
label variable hs "High School Degree"
label variable race "Race/Ethnicity"
label variable age "Age"
save "momslab1.dta", replace

use "dadslab.dta", clear
label variable hs "High School Degree"
label variable race "Race/Ethnicity"
label variable age "Age"
save "dadslab1.dta", replace

use "momslab1.dta", clear
append using "dadslab1.dta", generate(datascreen)
describe
```

(label eth already defined)

Contains data from momslab.dta

```
obs:      8
vars:     5
size:    136
```

27 Dec 2009 21:47

variable name	storage type	display format	value label	variable label
famid	float	%5.0g		Family ID
age	float	%5.0g		Mom's Age
race	float	%9.0g	eth	Mom's Ethnicity
hs	float	%15.0g	grad	Is Mom a HS Graduate?
datascreen	byte	%8.0g		

Sorted by:

Note: Dataset has changed since last saved.

file momslab1.dta saved

file dadslab1.dta saved

(label eth already defined)

Contains data from momslab1.dta

```
obs:      8
vars:     5
size:    136
```

11 Sep 2025 15:11

variable name	storage type	display format	value label	variable label
famid	float	%5.0g		Family ID
age	float	%5.0g		Age
race	float	%9.0g	eth	Race/Ethnicity
hs	float	%15.0g	grad	High School Degree
datascreen	byte	%8.0g		

Sorted by:

Note: Dataset has changed since last saved.

Note: we will use unique variable label names for merging.

3.2.3 Conflicting value labels

Using our previous files, we find that the value labels may also be incorrect when appending. The primary file (the open one) will supersede the values in the appended file.

Note: this will not throw an error and your data will append, but it might be confusing for yourself in the future or for a replicator. It is good practice to use neutral value labels and value label names.

Let's look at our files again. If you will notice that the value label names and the value labels will rename as the primary dataset's value label name and value labels' values.

```
use momslab, clear
codebook race hs

use dadslab, clear
codebook race hs

use "momsdadlab1.dta", replace
codebook race hs
```

race

```

              type:  numeric (float)
              label:  eth

              range:  [1,2]
unique values:  2                                units:  1
                                                    missing .:  0/4

tabulation:  Freq.    Numeric  Label
              2          1  Mom White
              2          2  Mom Black
```

hs

```

      type: numeric (float)
      label: grad

      range: [0,1]
unique values: 2
                        units: 1
                        missing .: 0/4

      tabulation: Freq.   Numeric   Label
                   1       0   Mom Not HS Grad
                   3       1   Mom HS Grad
```

race

```

      type: numeric (float)
      label: eth

      range: [1,2]
unique values: 2
                        units: 1
                        missing .: 0/4

      tabulation: Freq.   Numeric   Label
                   2       1   Dad White
                   2       2   Dad Black
```

hs

```

      type: numeric (float)
      label: hsgrad

      range: [0,1]
unique values: 2
                        units: 1
                        missing .: 0/4

      tabulation: Freq.   Numeric   Label
                   1       0   Dad Not HS Grad
                   3       1   Dad HS Grad
```

race

```

        type: numeric (float)
        label: eth

        range: [1,2]                      units: 1
unique values: 2                      missing .: 0/8

        tabulation: Freq.   Numeric   Label
                     4         1   Mom White
                     4         2   Mom Black

```

hs

```

        type: numeric (float)
        label: grad

        range: [0,1]                      units: 1
unique values: 2                      missing .: 0/8

        tabulation: Freq.   Numeric   Label
                     2         0   Mom Not HS Grad
                     6         1   Mom HS Grad

```

You will notice that the value label name in the dads file is `hsgrad`, while the moms file is `grad`. `Grad` supersedes the value label name `hsgrad` in the dads file. You will also notice that when you describe the data, a message for the value labels will say `eth` (which is the same for both), but all the data say Mom White or Mom Black.

Solution: Use neutral value labels and neutral value label names.

Solution: Label your variables and values after appending if the labels do not exist before appending.

3.2.4 Inconsistent variable coding

Another problem that will not throw an error, but it will cause problems are inconsistent variable coding. If we have a binary variable for high school degree or note, where one data set is 0-No and 1-Yes and the other is 1-No 2-Yes, an error will not be thrown when the datasets are appended. However, it will be a problem if you try to use a factor variable, since there conflicting and inconsistent coding.

Solution: Check your data, and check the data dictionaries of all the datasets.

Summarize and tabulate your data by the datasets after appending will help prevent this. When you find the issue, just recode the variables to be consistent.

```
use "momshs.dta", clear
append using "dads.dta", gen(datascreen)
list
tab hs datascreen
```

	famid	age	race	hs	datascreen
1.	3	24	2	2	0
2.	2	28	1	2	0
3.	4	21	1	1	0
4.	1	33	2	1	0
5.	1	21	1	0	1
6.	4	25	2	1	1
7.	2	25	1	1	1
8.	3	31	2	1	1

HS	datascreen		
Graduate?	0	1	Total
0	0	1	1
1	2	3	5
2	2	0	2
Total	4	4	8

Just recode the data to be consistent after finding the problem

```
use "momshs.dta", clear
recode hs (1=0) (2=1)
```

Or

```
replace hs=0 if hs == 1
replace hs=1 if hs == 2
append using "dads.dta", gen(datascreen)
tab hs datascreen
```

(2 real changes made)

(2 real changes made)

HS	datascreen		
Graduate?	0	1	Total
-----+			
0	2	1	3
1	2	3	5
-----+			
Total	4	4	8

3.2.5 Mixing variable types across datasets

When we try to append data of a different time an error will be thrown. We can use the `force` option to prevent the error, but as we have seen before this can cause numeric string variables with nonnumeric characters to become missing. This will cause data lose and additional measurement error.

```
cd "/Users/Sam/Desktop/Econ 645/Data/Mitchell"
use "moms.dta", clear
append using "dadstr.dta", generate(datascreen)
```

```
/Users/Sam/Desktop/Econ 645/Data/Mitchell
```

variable hs is float in master but str3 in using data

You could specify `append's force` option to ignore this numeric/string mismatch. This then be treated as if it contained numeric missing value.

Solution: Resolve the discrepancy in variable type before converting. Use the `destring` command on the string variable containing numeric data in string format. The option `force` with the `append` command forces string to numeric or numeric to string during the append. **Please note that using the option `force` with the `append` command may result in data loss if not properly analyzed beforehand.**

3.2.6 When we have a small dataset with clean numerics in string

Unfortunately, sometimes numeric variables are stored in a string type. We need to use the `destring` command to convert clean numerics that are stored as strings into a numeric type.

Solution:

```

cd "/Users/Sam/Desktop/Econ 645/Data/Mitchell"
use "dadstr.dta", clear
destring hs, replace
save "dadstrtemp.dta", replace

use "moms.dta", clear
append using "dadstrtemp.dta", gen(datascreen)
list, sepby(datascreen)

```

```
/Users/Sam/Desktop/Econ 645/Data/Mitchell
```

```
hs: all characters numeric; replaced as byte
```

```
file dadstrtemp.dta saved
```

	famid	age	race	hs	datascreen
1.	3	24	2	1	0
2.	2	28	1	1	0
3.	4	21	1	0	0
4.	1	33	2	1	0
5.	1	21	1	0	1
6.	4	25	2	1	1
7.	2	25	1	1	1
8.	3	31	2	1	1

3.2.7 When we have a large dataset with messy numerics and characters in string

When we have a small dataset, we can usually visually inspect the data before realizing that the numeric stored as a string is clean. With a large data file with thousands, tens of thousands, hundreds of thousands, or over a million observation, we cannot visually inspect the data. We need to use commands to summarize the data to look for problematic characteristics in numerics stored as strings. (E.g.: We want to get rid of \$)

Let's go back to **6.13 converting strings to numerics**.

In small datasets, this is easy to check and fix. HOWEVER, in large datasets, you will need different techniques. I found this on Statalist using regular ex-

pressions. As I have said before, regular expressions can be a pain, but they are powerful.

This statement below extracts the numerics from the string. <https://www.statalist.org/forums/forum/general-stata-discussion/general/967675-removing-non-numeric-characters-from-strings> *regexm()* looks for a numerics with “([0-9]+)” from the string *regexs()* looks for the nth part of the string.

```
cd "/Users/Sam/Desktop/Econ 645/Data/Mitchell"
use "dadstr.dta", clear
gen n = real(regexs(1)) if regexm(hs,"([0-9]+)")
list
```

```
/Users/Sam/Desktop/Econ 645/Data/Mitchell
```

	famid	age	race	hs	n
1.	1	21	1	0	0
2.	4	25	2	1	1
3.	2	25	1	1	1
4.	3	31	2	1	1

An example of how *regexm* and *regexs* work from the help file

```
clear
input str15 number
"(123) 456-7890"
"(800) STATAPC"
end

gen str newnum1 = regexs(1) if regexm(number, "^\\([0-9]+\\) (.*)")
gen str newnum2 = regexs(2) if regexm(number, "^\\([0-9]+\\) (.*)")
gen str newnum = regexs(1) + "-" + regexs(2) if regexm(number, "^\\([0-9]+\\) (.*)")
list number newnum
```

```

      number
1. "(123) 456-7890"
2. "(800) STATAPC"
3. end
```

number	newnum
"(123) 456-7890"	"123-456-7890"
"(800) STATAPC"	"800-STATAPC"

```

1. | (123) 456-7890    123-456-7890 |
2. | (800) STATAPC    800-STATAPC |
   +-----+

```

The point: The *regezm* command can be a lifesaver if you have numerical data stuck in string format with nonnumeric characters in a large file.

#Merging

Merging means we are append column based on key identifiers. Without a key variable or variables to merge upon, Stata will not be able to merge/join the datasets. Our main command to merge is *merge*

3.3 Merging: One-to-one

Appending is straightforward and relatively easy to check to make sure everything was appended well. Merging is a bit trickier, since additional problems can occur, along with different types of merging. Our command for merging is **merge**.

Note: Before discussing merging. It is **KEY** to have a key identifier variable that is common between two datasets if they will merge. Examples include personal id, firm id, state FIPS, county FIPS, zipcode, etc.

You may need multiple variables besides the identifier to properly merge. Let's say we want to merge state-level employment from the BLS QCEW with state-level GDP from BEA. We will likely need the county 2-digit FIPS code AND a time identifier, such as quarter or year. If the second identifier is missing we will not be able to properly merge the data.

Note: In Stata, there are two datasets when merging. One is called the *master* dataset and the other is called the *using* dataset.

Note: In Stata, we have four possible ways to merge: 1. 1:1 one-to-one merge 2. 1:m one-to-many merge (one in master and many in using) 3. m:1 many-to-one merge (many in master and one in using) 4. m:m many-to-many. DO NOT USE m:m!

From our moms and dads datasets our **KEY** variable to merge is family id (famid), and the moms1 dataset will be the master and the dads1 will be the using dataset

```

use moms1, clear
list

```

```

| famid  mage  mrace  mhs |
|-----|

```

1.		1	33	2	1	
2.		2	28	1	1	
3.		3	24	2	1	
4.		4	21	1	0	
+-----+						

There is only 1 observation per family, so we can do a 1-to-1 match.

```
merge 1:1 famid using dads1
list
```

Result	# of obs.
not matched	0
matched	4 (_merge==3)

	famid	mage	mrace	mhs	dage	drace	dhs	_merge
1.	1	33	2	1	21	1	0	matched (3)
2.	2	28	1	1	25	1	1	matched (3)
3.	3	24	2	1	31	2	1	matched (3)
4.	4	21	1	0	25	2	1	matched (3)

Notice: Our focus should be on `_merge`. If `_merge == 3`, then that means all of our matches worked. If `_merge == 1` or `_merge == 2`, then we have some non-merged observations. We may want this, or we might not expect this. Either way, it is a good idea to investigate.

Notice: Ironically enough, having two different variable names works well with `merge` compared to `append`. We now have two variables for `hs`, `race`, and `age`, but with `m` or `d` to distinguish moms and dads. You can easily reshape these data into a long format if necessary.

Problem: When we don't have perfect matches?

```
use "moms2.dta", clear
merge 1:1 famid using "dads2.dta"
```

Result	# of obs.
not matched	3
from master	2 (_merge==1)

```

      from using                                1  (_merge==2)

matched                                     2  (_merge==3)
-----

```

We can see that when we merge, a variable called `_merge` is created to identify which observations merged and which did not and why (only master, only using). We can tabulate the `_merge` variable that is created.

```
codebook _merge
tab _merge
```

```

_merge
-----

      type:  numeric (byte)
      label:  _merge

      range:  [1,3]
unique values:  3                      units:  1
                                         missing .:  0/5

      tabulation:  Freq.    Numeric  Label
                   2          1  master only (1)
                   1          2  using only (2)
                   2          3  matched (3)

      _merge |      Freq.    Percent    Cum.
-----+-----
master only (1) |          2      40.00     40.00
using only (2) |          1      20.00     60.00
matched (3) |          2      40.00    100.00
-----+-----
Total |          5     100.00

```

Not every family was in both datasets

```
sort famid
list famid mage mrace dage drace _merge
```

```

| famid  mage  mrace  dage  drace      _merge |
|-----+-----|
1. |    1    33     2    21     1  matched (3) |
2. |    2     .     .    25     1  using only (2) |
3. |    3    24     2     .     .  master only (1) |

```

```

4. |      4      21      1      25      2      matched (3) |
5. |      5      39      2      .      .      master only (1) |
+-----+

```

We have two matches between the data set, only 1 non-match that was only in the using data set (`_merge==2`), and 2 non-matches that were only in the master dataset (`_merge==1`).

It is a good idea to investigate why there were no matches between the master and using datasets. We may want that or we may not want depending upon the goal. We can use a qualifier to look at which variables were only in the master, using, and ones that matched

```

list famid mage mrace dage drace _merge if _merge==1
list famid mage mrace dage drace _merge if _merge==2
list famid mage mrace dage drace _merge if _merge==3

```

```

| famid  mage  mrace  dage  drace      _merge |
+-----+
2. |      3      24      2      .      .      master only (1) |
4. |      5      39      2      .      .      master only (1) |
+-----+

```

```

+-----+
| famid  mage  mrace  dage  drace      _merge |
+-----+
5. |      2      .      .      25      1      using only (2) |
+-----+

```

```

+-----+
| famid  mage  mrace  dage  drace      _merge |
+-----+
1. |      1      33      2      21      1      matched (3) |
3. |      4      21      1      25      2      matched (3) |
+-----+

```

Only famid 1 and 4 had observations in both datasets.

Let's say if we only want matched observations, and we are not concerned with unmatched observations, then we can keep only when `_merge == 3` and drop the non-matched observations.

```

keep if _merge == 3
list

```

(3 observations deleted)

	famid	mage	mrace	mhs	fr_moms2	dage	drace	dhs	fr_dads2	_merge
1.	1	33	2	1	1	21	1	0	1	matched (3)
2.	4	21	1	0	1	25	2	1	1	matched (3)

3.3.1 Potential Problem with 1-to-1 matching: duplicate ids

What if we have duplicate ids with a 1-to-1 matching?

```
use "momsdup.dta", clear
list
merge 1:1 famid using "dads2.dta"
```

	famid	mage	mrace	mhs	fr_moms2
1.	1	33	2	1	1
2.	3	24	2	1	1
3.	4	21	1	0	1
4.	4	39	2	0	1

variable famid does not uniquely identify observations in the master data
r(459);

r(459);

We have two observations for famid==4. If we use *merge 1:1 famid using "dads2.dta"*, it will throw an error, since famid does not uniquely identify units for matching. You would want to double check that famid is supposed to have two observations before proceeding.

3.3.2 1-to-1 Matching with more than one key variable

In our prior example we had duplicate for famid, and we might expect that if we had multiple family members. But, for 1-to-1 matching, we need a unique identifier(s) to properly match. Let's use kids1 for multiple kids for the same family.

```
use "kids1.dta", clear
sort famid kidid
list
```

	famid	kidid	kage	kfem
1.	1	1	3	1
2.	2	1	8	0
3.	2	2	3	1
4.	3	1	4	1
5.	3	2	7	0
6.	4	1	1	0
7.	4	2	3	0
8.	4	3	7	0

We now have a family id (famid) and kid id (kidid).

```
use "kidname.dta", clear
sort famid kidid
list
```

	famid	kidid	kname
1.	1	1	Sue
2.	2	1	Vic
3.	2	2	Flo
4.	3	1	Ivy
5.	3	2	Abe
6.	4	1	Tom
7.	4	2	Bob
8.	4	3	Cam

Let's merge using two key matching variable.

```
use "kids1.dta", clear
merge 1:1 famid kidid using "kidname.dta"
list
```

Result	# of obs.

```

not matched          0
matched              8  (_merge==3)
-----

+-----+
| famid  kidid  kage  kfem  kname  _merge |
+-----+
1. |    1    1    3    1    Sue  matched (3) |
2. |    2    1    8    0    Vic  matched (3) |
3. |    2    2    3    1    Flo  matched (3) |
4. |    3    1    4    1    Ivy  matched (3) |
5. |    3    2    7    0    Abe  matched (3) |
+-----+
6. |    4    1    1    0    Tom  matched (3) |
7. |    4    2    3    0    Bob  matched (3) |
8. |    4    3    7    0    Cam  matched (3) |
+-----+

```

3.4 Merging One-to-many

Sometimes we need to match file's observations to multiple observations in another data set. Maybe we have CPS data and we want to merge unemployment rates at the state-level to individuals units within those states. We have one values at the state-level for month m that needs to match multiple individuals in state s .

When we have multiple to one observation, we cannot use 1-to-1 merge. We need a 1:m merge. We can illustrate this with `moms1.dta` and `kids1.dta`. One mom may have multiple kids, so when we merge kids and moms data, we will need a 1:m merge.

```

use "moms1.dta", clear
list
use "kids1.dta", clear
list

```

```

+-----+
| famid  mage  mrace  mhs |
+-----+
1. |    1    33    2    1 |
2. |    2    28    1    1 |
3. |    3    24    2    1 |
4. |    4    21    1    0 |
+-----+

```

	famid	kidid	kage	kfem
1.	3	1	4	1
2.	3	2	7	0
3.	2	1	8	0
4.	2	2	3	1
5.	4	1	1	0
6.	4	2	3	0
7.	4	3	7	0
8.	1	1	3	1

Each kid and mom has a family id (famid) that we will use to match multiple kids to moms. Our moms have 1 observation while the kids have m observations. Since our moms have 1 observation and there are multiple kids, we need to align our 1:m properly. Since moms is the master the 1 is one the left of 1:m, while the using kids has multiple observations to family it is the m of 1:m.

```
use "moms1.dta", clear
merge 1:m famid using "kids1.dta"
list
```

Result	# of obs.
not matched	0
matched	8 (_merge==3)

	famid	mage	mrace	mhs	kidid	kage	kfem	_merge
1.	1	33	2	1	1	3	1	matched (3)
2.	2	28	1	1	2	3	1	matched (3)
3.	3	24	2	1	1	4	1	matched (3)
4.	4	21	1	0	1	1	0	matched (3)
5.	2	28	1	1	1	8	0	matched (3)
6.	3	24	2	1	2	7	0	matched (3)
7.	4	21	1	0	2	3	0	matched (3)
8.	4	21	1	0	3	7	0	matched (3)

If we were using kids as the master.

```
use "kids1.dta", clear
merge m:1 famid using "moms1.dta"
list, sepby(famid)
```

	Result	# of obs.
	not matched	0
	matched	8 (_merge==3)

	famid	kidid	kage	kfem	mage	mrace	mhs	_merge
1.	1	1	3	1	33	2	1	matched (3)
2.	2	2	3	1	28	1	1	matched (3)
3.	2	1	8	0	28	1	1	matched (3)
4.	3	1	4	1	24	2	1	matched (3)
5.	3	2	7	0	24	2	1	matched (3)
6.	4	1	1	0	21	1	0	matched (3)
7.	4	2	3	0	21	1	0	matched (3)
8.	4	3	7	0	21	1	0	matched (3)

If we tried to use 1 on the kids data and m on the moms side, then an error will be thrown saying that famid does not identify observation in the master dataset.

```
use "kids1.dta", clear
list
use moms1.dta, clear
list
use "kids1.dta", clear
merge 1:m famid using "moms1.dta"
```

If we use *merge 1:m famid using "moms1.dta"*, an error will be thrown.

	famid	kidid	kage	kfem
1.	3	1	4	1
2.	3	2	7	0
3.	2	1	8	0

4.		2	2	3	1	
5.		4	1	1	0	

6.		4	2	3	0	
7.		4	3	7	0	
8.		1	1	3	1	
+-----+						

+-----+						
		famid	mage	mrace	mhs	

1.		1	33	2	1	
2.		2	28	1	1	
3.		3	24	2	1	
4.		4	21	1	0	
+-----+						

variable famid does not uniquely identify observations in the master data
r(459);

r(459);

Reminder: 1. 1:m means “one-to-many” where one in master and many in
using 2. m:1 means “many-to-one” where many in master and one in using

Make sure your data are properly ordered in the merge command.

3.4.1 One-to-many merge with problems

Many times our data are not so clean for a perfect match, so what happens
when not all observations in both files have the same identifier (ex: famid)?
Let’s use data without all of the same identifiers.

```
use "moms2.dta", clear
list
use "kids2.dta", clear
list, sepby(famid)
```

		famid	mage	mrace	mhs	fr_moms2	

1.		1	33	2	1	1	
2.		3	24	2	1	1	
3.		4	21	1	0	1	
4.		5	39	2	0	1	

	famid	kidid	kage	kfem
1.	2	2	3	1
2.	2	1	8	0
3.	3	2	7	0
4.	3	1	4	1
5.	4	2	3	0
6.	4	3	7	0
7.	4	1	1	0

Merge 1-to-many

```
use "moms2.dta", clear
merge 1:m famid using "kids2.dta"
```

Result	# of obs.
not matched	4
from master	2 (_merge==1)
from using	2 (_merge==2)
matched	5 (_merge==3)

We have 5 matched observations and 4 unmatched observations. 2 observations were only in the moms dataset and 2 observations were in the kids dataset.

```
tab _merge
sort famid kidid
list, sepby(famid)
```

_merge	Freq.	Percent	Cum.
master only (1)	2	22.22	22.22
using only (2)	2	22.22	44.44
matched (3)	5	55.56	100.00
Total	9	100.00	

	famid	mage	mrace	mhs	fr_moms2	kidid	kage	kfem	_merge
1.	1	33	2	1	1	.	.	.	master only (1)
2.	2	.	.	.	.	1	8	0	using only (2)
3.	2	.	.	.	.	2	3	1	using only (2)
4.	3	24	2	1	1	1	4	1	matched (3)
5.	3	24	2	1	1	2	7	0	matched (3)
6.	4	21	1	0	1	1	1	0	matched (3)
7.	4	21	1	0	1	2	3	0	matched (3)
8.	4	21	1	0	1	3	7	0	matched (3)
9.	5	39	2	0	1	.	.	.	master only (1)

When `_merge==1` we have missing observations in the kids variables, and when `_merge==2` we have missing observations in the moms variables. Non-matched data

```
list if _merge == 1 | _merge == 2, sepby(famid)
```

	famid	mage	mrace	mhs	fr_moms2	kidid	kage	kfem	_merge
1.	1	33	2	1	1	.	.	.	master only (1)
2.	2	.	.	.	.	1	8	0	using only (2)
3.	2	.	.	.	.	2	3	1	using only (2)
9.	5	39	2	0	1	.	.	.	master only (1)

Matched data

```
list if _merge == 3, sepby(famid)
```

	famid	mage	mrace	mhs	fr_moms2	kidid	kage	kfem	_merge
4.	3	24	2	1	1	1	4	1	matched (3)
5.	3	24	2	1	1	2	7	0	matched (3)

6.		4	21	1	0	1	1	1	0	matched (3)	
7.		4	21	1	0	1	2	3	0	matched (3)	
8.		4	21	1	0	1	3	7	0	matched (3)	

3.5 Merging multiple datasets

Sometimes we need to merge more than 2 datasets together. The examples in the book have `nogenerate` in the `merge` command. **I don't recommend this**, and after you have inspected your first merge and are satisfied with the results use the `drop` command to drop `_merge`, and then proceed with your second merge.

Let's say we have four datasets: "moms2.dta", "momsbest2.dta", "dads2.dta", "dadsbest2.dta".

```
use "moms2.dta", clear
merge 1:1 famid using "momsbest2.dta"
sort famid
list, sepby(famid)
```

Result	# of obs.
not matched	3
from master	2 (_merge==1)
from using	1 (_merge==2)
matched	2 (_merge==3)

	famid	mage	mrace	mhs	fr_moms2	mbage	fr_mo~t2	_merge
1.	1	33	2	1	1	.	.	master only (1)
2.	2	.	.	.	.	29	1	using only (2)
3.	3	24	2	1	1	23	1	matched (3)
4.	4	21	1	0	1	37	1	matched (3)
5.	5	39	2	0	1	.	.	master only (1)

Inspect the merge

```
tab _merge
```

_merge	Freq.	Percent	Cum.
master only (1)	2	40.00	40.00
using only (2)	1	20.00	60.00
matched (3)	2	40.00	100.00
Total	5	100.00	

You may want another variable to inspect in the tabulation, which is helpful with large datasets where you cannot eyeball every observation. For example:

```
tab mage _merge
```

Age	_merge	Total
21	0 1	1
24	0 1	1
33	1 0	1
39	1 0	1
Total	2 2	4

Drop the `_merge` after successful inspection. If you are doing multiple merging, *do not forget* to drop `*_merge*` or Stata will throw an error.

```
drop _merge
```

Merge the 3rd dataset and inspect

```
merge 1:1 famid using "dads2.dta"
tab _merge
```

Result	# of obs.
not matched	2
from master	2 (_merge==1)
from using	0 (_merge==2)

matched		3 (_merge==3)		

	_merge	Freq.	Percent	Cum.
-----+-----				
master only (1)		2	40.00	40.00
matched (3)		3	60.00	100.00
-----+-----				
Total		5	100.00	

Drop the 2nd merge for 3rd merge

```
drop _merge
```

Merge the 4th dataset and inspect the merge

```
merge 1:1 famid using "dadsandbest.dta"
sort famid
list famid fr_*, sepby(famid)
```

Result	# of obs.

not matched	1
from master	1 (_merge==1)
from using	0 (_merge==2)
matched	4 (_merge==3)

+-----+	
	famid fr_moms2 fr_mo~t2 fr_dads2 fr_da~t2

1.	1 1 . 1 1

2.	2 . 1 1 1

3.	3 1 1 . 1

4.	4 1 1 1 1

5.	5 1 . . .
	+-----+

Drop for 4th merge

```
drop _merge
```

Now add a 4th merge but with a 1-to-many, and inspect the merge

```
merge 1:m famid using "kidname.dta"
tab _merge
list famid fr_* kidname, sepby(famid)
```

Result	# of obs.		

not matched	1		
from master	1	(_merge==1)	
from using	0	(_merge==2)	
matched	8 (_merge==3)		

_merge	Freq.	Percent	Cum.

master only (1)	1	11.11	11.11
matched (3)	8	88.89	100.00

Total	9	100.00	

```
variable kidname not found
r(111);
```

```
r(111);
```

Mitchell suggests a user-contribution command called *dmtablist*. You must have Stata 16 or higher, so I'm unable to demonstrate it.

3.5.1 Update mergers

There is an interesting update option with the merge if for some reason, you wanted to check an older version of your data. It will replace the data in your master file with data in your using file. I have never used this set of options, but they may have value in future situations.

```
cd "/Users/Sam/Desktop/Econ 645/Data/Mitchell"
use moms5, clear
list
```

/Users/Sam/Desktop/Econ 645/Data/Mitchell

```

+-----+
| famid  mage  mrace  mhsgrad |
+-----+
1. |      1      .      2      1 |
2. |      2     82      .      1 |
3. |      3     24      2      . |
4. |      4     21      1      0 |
+-----+

```

Here is an updated file with error corrections and previously missing data

```

use moms5fixes, clear
list

```

```

| famid  mage  mrace  mhsgrad |
+-----+
1. |      1     33      .      . |
2. |      2     28      1      . |
3. |      3      .      .      1 |
+-----+

```

If we use the update option, then it will find matching data, missing data to update, and conflicting data, which are data that match on the key variable, but are of different values.

We'll need to inspect the merge: 1. We have 1 observation famid==4 that there are no corresponding observations in our using data 2. We have 2 missing data in our master that are updated with data from using 3. We have 1 conflict data between the master and using datasets: 82 vs 28.

```

use moms5, clear
merge 1:1 famid using moms5fixes, update
sort famid
list

```

```

Result                                     # of obs.
-----
not matched                               1
  from master                             1  (_merge==1)
  from using                              0  (_merge==2)

matched                                   3
  not updated                             0  (_merge==3)

```

```

missing updated                2  (_merge==4)
nonmissing conflict            1  (_merge==5)
-----

+-----+
| famid  mage  mrace  mhsgrad                _merge |
+-----+
1. |      1    33      2      1      missing updated (4) |
2. |      2    82      1      1  nonmissing conflict (5) |
3. |      3    24      2      1      missing updated (4) |
4. |      4    21      1      0      master only (1) |
+-----+

```

If we want to update the conflicting data, then we need to the *replace* option along with the *update* option. Next, we'll need to inspect the merge. Remember, if it is a large data set, then the *tab* command by variables may be more appropriate than the *list* command.

```

use moms5, clear
merge 1:1 famid using moms5fixes, update replace

sort famid
list

```

```

Result                                # of obs.
-----
not matched                            1
  from master                          1  (_merge==1)
  from using                           0  (_merge==2)

matched                                3
  not updated                          0  (_merge==3)
  missing updated                      2  (_merge==4)
  nonmissing conflict                  1  (_merge==5)
-----

+-----+
| famid  mage  mrace  mhsgrad                _merge |
+-----+
1. |      1    33      2      1      missing updated (4) |
2. |      2    28      1      1  nonmissing conflict (5) |
3. |      3    24      2      1      missing updated (4) |
4. |      4    21      1      0      master only (1) |
+-----+

```

3.5.2 Merging Additional options

There are some additional options in `merge`, which may be of interest that we will cover.

`keepusing()`

The `keepusing()` option can be helpful when merging two datasets with hundreds of variables. Our data sets here are small, but the CPS, ACS, etc. can have hundreds of variables and we may only want a few additional variables from a using dataset

Let's say we only want dads age and dads race from our using dataset. We can use the `keepusing()` option to keep dage drace

```
cd "/Users/Sam/Desktop/Econ 645/Data/Mitchell"
use dads1, clear
list
use moms1, clear
merge 1:1 famid using dads1, keepusing(dage drace)
list
```

We do not keep dhs after the merge since we specify dage and drace with the `keepusing()` option

`assert()`

Another interesting option is `assert()`, which could be useful in certain situations. If we specify `assert(match)`, Stata will throw an error if all observations are not matched. This may or may not be helpful when inspecting the data post-merge. An option of `assert(match master)` makes sure that all merges are matched or from the original dataset.

More information is on pages 247-248.

Other options

I do not recommend using `noreport` or `nogenerate` options, since these are essential for inspecting your merges.

The `generate` option is just basically a rename of `__merge`

I don't recommend the `keep()` option either. You should inspect your data and then when you are satisfied, you can use the drop command with a qualifier using `__merge`: drop if `__merge == 1 | __merge == 2`

3.6 Merging Problems

Merging can be trickier than appending. Appending is fairly straightforward, but some of the topics are similar

Common variable names

This is a similar problem to append, but we need different variable names instead of the same variable names. If we have common or the same variable names with merge, then we will lose data.

In our example we have similar columns of data, but with different names except for mom's age and dad's age which are both named "age". For example the race column in moms data is called race, while the race column in dads data is called eth. But, both datasets have age named as age.

```
cd "/Users/Sam/Desktop/Econ 645/Data/Mitchell"
use moms3, clear
list
use dads3, clear
list
use moms3, clear
merge 1:1 famid using dads3
list
```

```
/Users/Sam/Desktop/Econ 645/Data/Mitchell
```

	famid	age	race	hs
1.	1	33	2	1
2.	2	28	1	1
3.	3	24	2	1
4.	4	21	1	0

	famid	age	eth	gradhs
1.	1	21	1	0
2.	2	25	1	1
3.	3	31	2	1
4.	4	25	2	1

Result	# of obs.
not matched	0
matched	4 (_merge==3)

	famid	age	race	hs	eth	gradhs	_merge
1.	1	33	2	1	1	0	matched (3)
2.	2	28	1	1	1	1	matched (3)
3.	3	24	2	1	2	1	matched (3)
4.	4	21	1	0	2	1	matched (3)

Even though our merge is successful, but we lost dads age. Please note that even when a merge appears successful, we still need to inspect our data to look for any problem such as lost data. This easy to notice in a small dataset, but what about datasets with hundreds of variables? We should inspect our data systematically beforehand to prevent this problem.

Solution: There is a helpful command called *cf* (compare files) that can detect variables that are common/same and different between datasets. Once detected, rename the variable to a name that would make sense e.g: momsage.

```
use moms3, clear
cf _all using dads3, all verbose
```

```
famid: match
age: 4 mismatches
      obs 1. 33 in master; 21 in using
      obs 2. 28 in master; 25 in using
      obs 3. 24 in master; 31 in using
      obs 4. 21 in master; 25 in using
race: does not exist in using
hs:   does not exist in using
r(9);

r(9);
```

From the *cf* command, we want to compare variables and we find that race and hs are not in the master file, but age is and there are 4 mismatched observations. We can easily enough rename our age variable in the master before performing our merge. Our all option will compare all the variables, but if we don't use this option, only mismatched will appear. There is a *verbose* option that will

give a detailed listing of each observation that differs. *verbose* is probably not practical when merging data between large datasets.

Notice: our key variable famid does not appear as a mismatch.

Remember, In append, when we had different variable names, we would create new and inconsistent columns of data. In merge we want new variable names to preserve all of our data.

Mitchell recommends that you check out a blog on Merges Gone Bad: <https://blog.stata.com/2011/04/18/merging-data-part-1-merges-gone-bad/>

`search merges gone bad`

Same value label names

This is a similiar but slightly different problem that we saw in append. When there are two value labels of the same name, the master value label name will overwrite the using value label name. A message will pop up, but an error will not be thrown.

In our example, high school has label dhs and label mhs so they are unique, but race is common between moms4 and dads4, and when merging the master label values will overwrite the using.

```
cd "/Users/Sam/Desktop/Econ 645/Data/Mitchell"
use moms4, clear
merge 1:1 famid using dads4
list famid mrace drace mhs dhs
```

```
/Users/Sam/Desktop/Econ 645/Data/Mitchell
```

(label race already defined)

	Result	# of obs.
	not matched	0
	matched	4 (_merge==3)

	famid	mrace	drace	mhs	dhs
1.	1	Mom Black	Mom White	Mom HS Grad	Dad Not HS Grad
2.	2	Mom White	Mom White	Mom HS Grad	Dad HS Grad

3.		3	Mom Black	Mom Black	Mom HS Grad	Dad HS Grad	
4.		4	Mom White	Mom Black	Mom Not HS Grad	Dad HS Grad	

+-----+

A message saying “label race already defined”)

Solution: create a new label values for dad’s race.

```
label define drace1 1 "White" 2 "Black"
label values drace drace1
list famid mrace drace mhs dhs
```

		famid	mrace	drace	mhs	dhs	

1.		1	Mom Black	1	Mom HS Grad	Dad Not HS Grad	
2.		2	Mom White	1	Mom HS Grad	Dad HS Grad	
3.		3	Mom Black	2	Mom HS Grad	Dad HS Grad	
4.		4	Mom White	2	Mom Not HS Grad	Dad HS Grad	

+-----+

Conflicts in key variables

Solutions: 1. Precheck all variable names with `describe` to prevent this (but this may be unreasonable in large datasets), and then label value labels of the common value labels to a more generic value label. 2. An alternative could be to relabel your data after a merge. 3. Mitchell recommends the `precombine` command which is available in later versions of Stata.

Note: your key variables need to be in the same type numerics or strings. Please check your key variables before merging.

m:m matching

You need to be careful when doing m:m matching, and I don’t think Mitchell even talks about this. It is preferable to have your main dataset as your unique observations (your 1), and your using having multiple observation (your m). There could be situations where you might need it, but it shouldn’t be your default go to for merge

```
cd "/Users/Sam/Desktop/Econ 645/Data/Mitchell"
use moms1, clear
append using dads1
sort famid
list
```

/Users/Sam/Desktop/Econ 645/Data/Mitchell

	famid	mage	mrace	mhs	dage	drace	dhs
1.	1	.	.	.	21	1	0
2.	1	33	2	1	.	.	.
3.	2	.	.	.	25	1	1
4.	2	28	1	1	.	.	.
5.	3	.	.	.	31	2	1
6.	3	24	2	1	.	.	.
7.	4	.	.	.	25	2	1
8.	4	21	1	0	.	.	.

An example of a problem with m:m merging. We now have multiple famid ids: one for moms, one for dads. And now if we want to merge kids data, we try a m:m matching since we have multiple parents and multiple kids.

```
merge m:m famid using kids1
list
```

Result	# of obs.
not matched	0
matched	9 (_merge==3)

	famid	mage	mrace	mhs	dage	drace	dhs	kidid	kage	kfem	_merge
1.	1	.	.	.	21	1	0	1	3	1	matched
2.	1	33	2	1	.	.	.	1	3	1	matched
3.	2	.	.	.	25	1	1	1	8	0	matched
4.	2	28	1	1	.	.	.	2	3	1	matched
5.	3	.	.	.	31	2	1	2	7	0	matched
6.	3	24	2	1	.	.	.	1	4	1	matched
7.	4	.	.	.	25	2	1	2	3	0	matched
8.	4	21	1	0	.	.	.	3	7	0	matched
9.	4	21	1	0	.	.	.	1	1	0	matched

We need to be careful, and observe our data for problems. We have a duplicative kidid since famid==1 only has one kid. We have a duplicative mom in

famid==4. This is likely very problematic.

Solution: A better approach would be to merge moms and dads data, and then merge the kids data with a 1:m. A *joinby* might be preferable here, which will be discussed next

3.7 Joining datasets

The *joinby* command is similar to the merge command but there are some differences.

There are different types of joinby that can be changed by the unmatched() option. There is inner joinby where unmatched() is left off. This is only the matched observations. There is a left joinby where unmatched(master) is added as an option, and we keep all master observation matched or unmatched and drop all unmatched using observations. There is a right joinby where unmatched(using) is added as an option, and we keep all using observations matched or unmatched and drop all unmatched master observation.

joinby command only keeps matched observations by default, but this can be very problematic. I suggest using the unmatched(master) option as it is a left join. You should probably keep your observations in at least one dataset.

I personally recommend that you use merge instead of joinby, since you have more inspection diagnostics to make sure your data merged properly. But there may be situations where a left-join is a better option instead of a m:m merge. This usually deals with multiple observations for an id variable such as moms and dads and multiple kids.

We have multiple kids for each family ranging from 1 to 3.

```
cd "/Users/Sam/Desktop/Econ 645/Data/Mitchell"
use kidname, clear
sort famid kname
list, sepby(famid)
```

```
/Users/Sam/Desktop/Econ 645/Data/Mitchell
```

```
+-----+
| famid  kidid  kname |
+-----+
1. |      1      1    Sue |
+-----+
2. |      2      2    Flo |
3. |      2      1    Vic |
+-----+
```

4.		3	2	Abe	
5.		3	1	Ivy	

6.		4	2	Bob	
7.		4	3	Cam	
8.		4	1	Tom	

		+-----+			

We have two parents in each family

```
use parname, clear
sort famid
list
```

		famid	mom	age	race	pname	

1.		1	0	21	1	Sam	
2.		1	1	33	2	Lil	
3.		2	0	25	1	Nik	
4.		2	1	28	1	Ula	
5.		3	0	31	2	Al	

6.		3	1	24	2	Ann	
7.		4	0	25	2	Ted	
8.		4	1	21	1	Bev	

		+-----+					

We'll look at an *inner join* first

```
joinby famid using kidname
sort famid kname pname
list, sepby(famid kidid)
```

		famid	mom	age	race	pname	kidid	kname	

1.		1	1	33	2	Lil	1	Sue	
2.		1	0	21	1	Sam	1	Sue	

3.		2	0	25	1	Nik	2	Flo	
4.		2	1	28	1	Ula	2	Flo	

5.		2	0	25	1	Nik	1	Vic	
6.		2	1	28	1	Ula	1	Vic	

7.		3	0	31	2	Al	2	Abe	
8.		3	1	24	2	Ann	2	Abe	

9.		3	0	31	2	Al	1	Ivy	
10.		3	1	24	2	Ann	1	Ivy	

11.		4	1	21	1	Bev	2	Bob	
12.		4	0	25	2	Ted	2	Bob	

13.		4	1	21	1	Bev	3	Cam	
14.		4	0	25	2	Ted	3	Cam	

15.		4	1	21	1	Bev	1	Tom	
16.		4	0	25	2	Ted	1	Tom	
-----+									

Notice we have a family id, but also a parent id in the mom variable (0,1). We can have a **left-join**, but it will be the same as an **inner join**. This might be preferable, since not all parents may have kids and we would want to keep their data.

We'll look at a **left-join**

```
use parname, clear
joinby famid using kidname, unmatched(master)
sort famid kname pname
list, sepby(famid kidid)
```

		famid	mom	age	race	pname		_merge	kidid	kname	

1.		1	1	33	2	Lil	both in master and using data		1	Sue	
2.		1	0	21	1	Sam	both in master and using data		1	Sue	

3.		2	0	25	1	Nik	both in master and using data		2	Flo	
4.		2	1	28	1	Ula	both in master and using data		2	Flo	

5.		2	0	25	1	Nik	both in master and using data		1	Vic	
6.		2	1	28	1	Ula	both in master and using data		1	Vic	

7.		3	0	31	2	Al	both in master and using data		2	Abe	
8.		3	1	24	2	Ann	both in master and using data		2	Abe	

9.		3	0	31	2	Al	both in master and using data		1	Ivy	
10.		3	1	24	2	Ann	both in master and using data		1	Ivy	

11.		4	1	21	1	Bev	both in master and using data		2	Bob	

12.		4	0	25	2	Ted	both in master and using data	2	Bol

13.		4	1	21	1	Bev	both in master and using data	3	Car
14.		4	0	25	2	Ted	both in master and using data	3	Car

15.		4	1	21	1	Bev	both in master and using data	1	Tor
16.		4	0	25	2	Ted	both in master and using data	1	Tor

3.8 Crossing datasets

If you are interested in cross data, it can be found on pages 255-257.

The cross command matches each observation in the master dataset to the using dataset.

In our moms and dads data set, we have 4 moms and 4 dads. The cross command will match every possible combination, so we have a 4x4 outcome. This might be of interest, and have some usefulness with combination and permutations, but I have never personally used this.

```
cd "/Users/Sam/Desktop/Econ 645/Data/Mitchell"
use moms1, clear
cross using dads1
sort famid
list, sepby(famid)
```

/Users/Sam/Desktop/Econ 645/Data/Mitchell

+-----+								
		famid	mage	mrace	mhs	dage	drace	dhs

1.		1	33	2	1	25	1	1
2.		1	33	2	1	31	2	1
3.		1	33	2	1	21	1	0
4.		1	33	2	1	25	2	1

5.		2	28	1	1	31	2	1
6.		2	28	1	1	25	2	1
7.		2	28	1	1	25	1	1
8.		2	28	1	1	21	1	0

9.		3	24	2	1	21	1	0
10.		3	24	2	1	31	2	1
11.		3	24	2	1	25	2	1

12.		3	24	2	1	25	1	1	

13.		4	21	1	0	25	1	1	
14.		4	21	1	0	21	1	0	
15.		4	21	1	0	31	2	1	
16.		4	21	1	0	25	2	1	
		+-----+							

Mitchell, N.M. (2020). *Data Management Using Stata: A Practical Handbook*. (2nd ed.). Stata Press.

UCLA Advanced Research Computing. (2025). *Stata Learning Modules*. Retrieved from <https://stats.oarc.ucla.edu/stata/modules/>

Wooldridge (2009). *Introductory Econometrics: A Modern Approach*. (4th ed.). South-Western Cengage Learning.